PROOF-CARRYING AUTHORIZATION / EXACT EFFECTS

Auths-Proof: Proof-Carrying Authorization for Exact, Recoverable Effects

bordumb · bordumbb@gmail.com

15 August 2026

ABSTRACT

Modern systems authenticate people and machines well, yet still struggle to answer the question that matters at an effect boundary: may this actor perform this exact action, under these exact limits, now? Auths-Proof is a proof-carrying authorization system for that boundary. An actor carries portable evidence of narrowly delegated authority; a verifier combines it with local trust and policy; and a production runtime turns an authorized canonical action into one durable, recoverable, auditable effect.

The design separates three facts that ordinary authorization stacks often collapse: whether an action is authorized, whether a provider effect may have happened, and what fresh observation establishes afterward. Its offline Rust kernel is deterministic and fail-closed. Delegation is a product order over eleven authority dimensions, three-valued composition preserves trustworthy uncertainty, and terminal budget coverage refuses an action that omits a requested budget beneath a bounded ceiling. Effectful requests enter a bounded durable lifecycle before provider I/O. A client-generated recovery reference makes response loss reconcilable, while profile-owned workers keep provider commands, credentials, observations, and receipt meaning closed.

The open production substrate combines the verified kernel, PostgreSQL lifecycle state, KMS or PKCS#11 custody, three exact-effect profiles, signed decision and execution receipts, bounded OTLP export, Prometheus projection, and thin Rust, TypeScript, and Python clients. Formal assurance connects readable Lean semantics to the pure Rust decisions used in production through qualified Charon/Aeneas translation. Generated contracts, exhaustive finite vectors, mutation controls, Kani harnesses, cross-language conformance, and adversarial corpus tests make semantic drift a release failure.

Auths-Proof does not assume that networks or external providers are atomic. It provides exact authorization, exactly-once durable admission, honest possible-effect states, and profile-specific reconciliation. The result is a small semantic center surrounded by explicit operational machinery: portable authority without portable trust, powerful delegation without amplification, and automation whose effects remain explainable after crashes, timeouts, and retries.

1. Introduction

Authentication establishes who or what controls a credential. Authorization establishes what that principal may do. Execution establishes whether a real-world effect occurred. These are related judgments, but they are not the same judgment.

A valid signature does not authorize a database update. A workload certificate does not authorize an infrastructure deployment. A human approval does not prove that a provider applied the approved change, and an HTTP timeout does not prove that it did not. Systems that collapse these boundaries eventually make one of two unsafe statements: “the caller is authenticated, therefore proceed,” or “the request failed, therefore retry.”

> **Auths-Proof carries exact authority to a verifier, derives one canonical action under verifier-local trust, admits any effect durably, and records only what authorization and observation can actually establish.**

This design draws on capability systems, trust management, proof-carrying authorization, and formal refinement (Dennis and Horn 1966; Blaze, Feigenbaum, and Lacy 1996; Appel and Felten 1999; Bauer, Schneider, and Felten 2002). Its distinguishing concern is the full path from portable delegated authority to a recoverable external effect. Figure 1 shows that path.

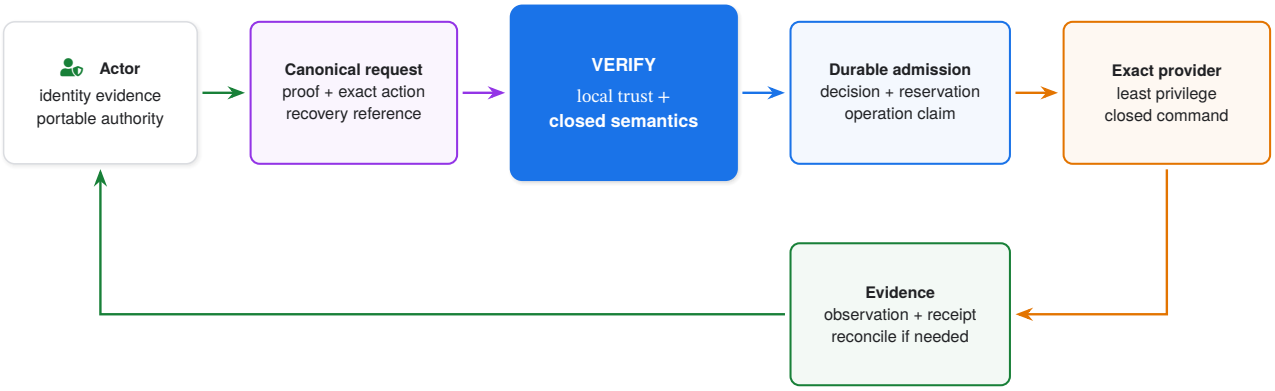

Figure 1: The authority-to-effect loop. Authorization is decided before provider entry. Provider outcome is observed afterward. A lost response follows the durable recovery path rather than becoming a blind retry.

1.1 Design goals

Auths-Proof is organized around six goals.

Exactness. The authorized action is a canonical typed value. The executor cannot substitute a request that merely resembles it.

Attenuation. Delegation may preserve or narrow authority, never widen it. The local verifier chooses trust roots, registries, time, assurance, and hard limits.

Fail-closed uncertainty. Missing trustworthy evidence is distinct from denial but never permits execution. A possible provider effect is distinct from both success and proven non-effect.

Durable effect safety. Every effectful operation is admitted and reserved before credentials or provider I/O. Timeouts and response loss retain a client-known recovery path.

Semantic ownership. Rust owns protocol meaning. TypeScript and Python project that meaning; profiles own their provider-specific actions and lifecycles; transports and telemetry cannot upgrade authority.

Auditable correspondence. Formal models, shipping predicates, generated artifacts, fixtures, and release gates change together.

1.2 Contributions

This paper makes seven contributions.

1. A portable proof format evaluated under verifier-local trust rather than proof-supplied trust.
2. A closed, eleven-dimensional delegation order with exact terminal coverage and critical-extension preservation.
3. Three-valued authorization composition that preserves trustworthy uncertainty through conjunction, disjunction, and thresholds.
4. A durable exact-effect lifecycle with bounded admission, client-known recovery, honest unknown outcomes, and profile-owned reconciliation.
5. Separate signed decision and execution receipts with bounded, authorized disclosure.
6. A self-hostable production architecture with qualified storage, custody, profiles, operations, and language projections.
7. A refinement and conformance chain that connects readable semantics to the production Rust decisions and makes negative tests part of release gating.

1.3 Scope and terminology

The word *proof* has three meanings in this system:

- an **authorization proof** is untrusted portable protocol evidence;
- a **Lean proof** is a theorem checked by the Lean kernel; and
- a **Kani proof harness** is a bounded symbolic check over shipping Rust.

An **authority** is a bounded right to perform actions. An **action** is a profile-canonical description of one proposed effect. A **decision** is the kernel's authorization result. An **effect** is a provider mutation or durable external decision.

A **receipt** is attested evidence about a decision or execution; it is never itself authority.

2. System model and architecture

2.1 Five layers, one semantic direction

Auths-Proof is a layered monolith of packages. Keeping the implementation in one repository allows a protocol change to update the core, transports, products, bindings, fixtures, proofs, and release evidence atomically. Dependency direction prevents that convenience from becoming semantic coupling.

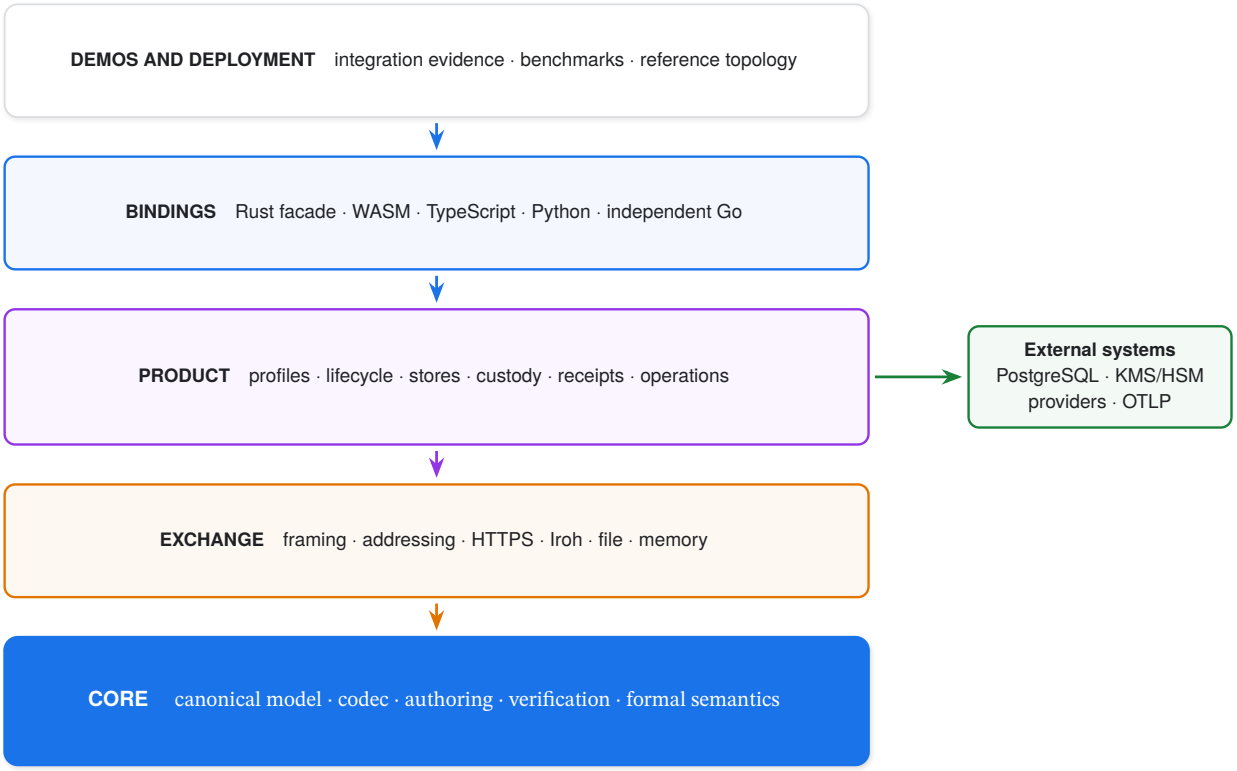

Figure 2: Layered system architecture. Dependencies move toward the offline core. External state and I/O remain in product adapters. Bindings expose semantics but never define them.

The core has no network, filesystem, process, environment, mutable database, or wall-clock dependency. Exchange moves opaque proof bytes but cannot make an invalid proof valid. Product packages add stateful policy and effects without redefining verification. Bindings project Rust-owned results. Demos and deployments prove integration behavior but are not sources of production semantics.

2.2 Actors and trust boundaries

The system distinguishes five roles:

- the **issuer**, who creates or delegates authority;
- the **actor**, who proposes an exact action and presents proof;
- the **verifier**, who owns trust roots, registries, time, status, assurance, and resource limits;
- the **runtime**, which durably admits an authorized effect; and
- the **provider**, which performs or reports the external effect.

The portable proof contains signed grants, action commitments, evidence, bindings, and an authorization plan. It does not contain verifier trust. A proof may identify a trust root, but only the local trusted context decides whether that root is accepted. It may carry status evidence, but only local policy decides the accepted method and freshness. This is verifier sovereignty.

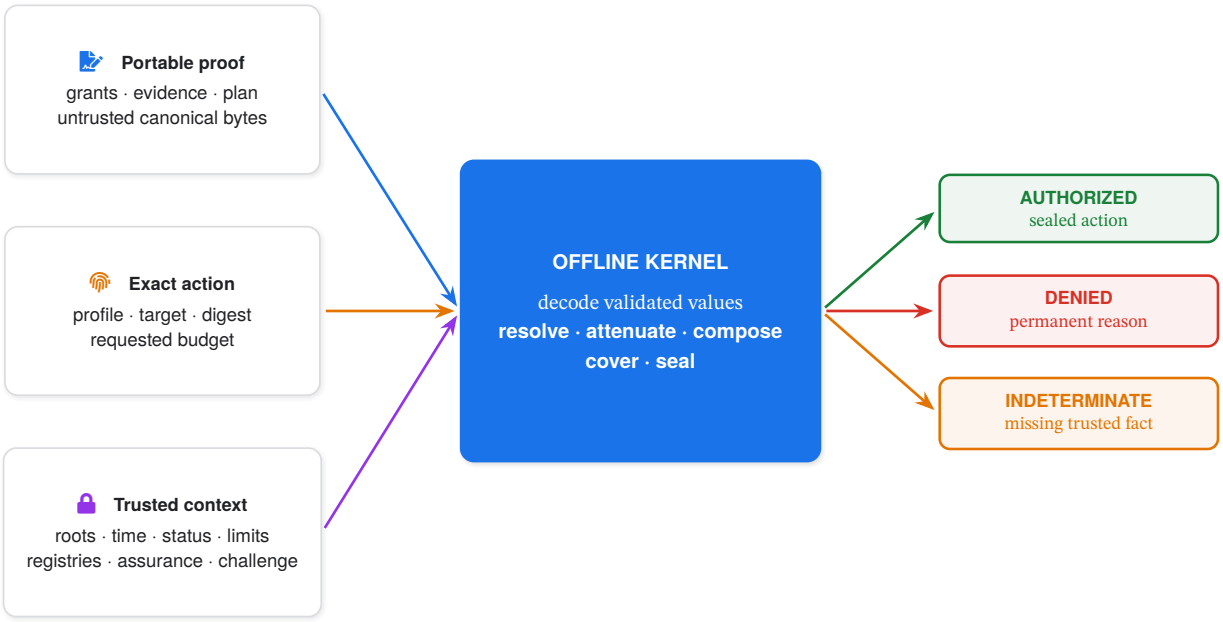

Figure 3: **The verifier's narrow waist.** Identity and evidence contribute facts. Only the local authority computation can produce the sealed canonical action accepted by the effect runtime.

2.3 Threat model

Public inputs are adversarial. An attacker may submit malformed or non-canonical bytes, oversized collections, ambiguous encodings, invalid signatures, widened delegations, stale evidence, configuration mismatches, replayed actions, forged recovery references, duplicate requests, or provider responses designed to confuse reconciliation.

The attacker may control a valid principal credential without holding the required authority. Networks may delay, duplicate, reorder, truncate, or lose requests and responses. Processes may crash at any instruction boundary. Providers may accept a command and lose the response, return an ambiguous result, or become temporarily inconsistent.

The design does not assume provider atomicity. It assumes the selected cryptographic primitives, local trust configuration, qualified storage and custody implementations, compiler/toolchain basis, and profile-specific provider observations satisfy their published contracts.

3. Authorization semantics

3.1 Authority as a closed product order

An effective authority is described by a trust root, current subject, remaining delegation depth, and a semantic scope. The scope contains profile selection, permissions, inclusive validity, audiences, action-body constraints, optional budget ceilings, status policy, assurance policy, and critical extensions.

Delegation is accepted only when all eleven checks hold:

$$\begin{aligned}&\text{root preserved} \wedge \text{depth strictly decreases} \wedge \text{profile narrows}\\ &\wedge \text{permissions narrow} \wedge \text{validity narrows} \wedge \text{audiences narrow}\\ &\wedge \text{action constraint narrows} \wedge \text{budget narrows} \wedge \text{status does not weaken}\\ &\wedge \text{assurance does not weaken} \wedge \text{critical extensions are preserved.}\end{aligned}$$

Finite permissions and audiences narrow by subset. Validity narrows by interval containment. Action constraints move from any body to an allowed digest set to one exact digest. Budgets may decrease but not disappear. Snapshot status may require fresher evidence but may not change method. Assurance is invariant. Critical extensions are equality-preserved because an unaware delegate must not strip a constraint it is required to understand.

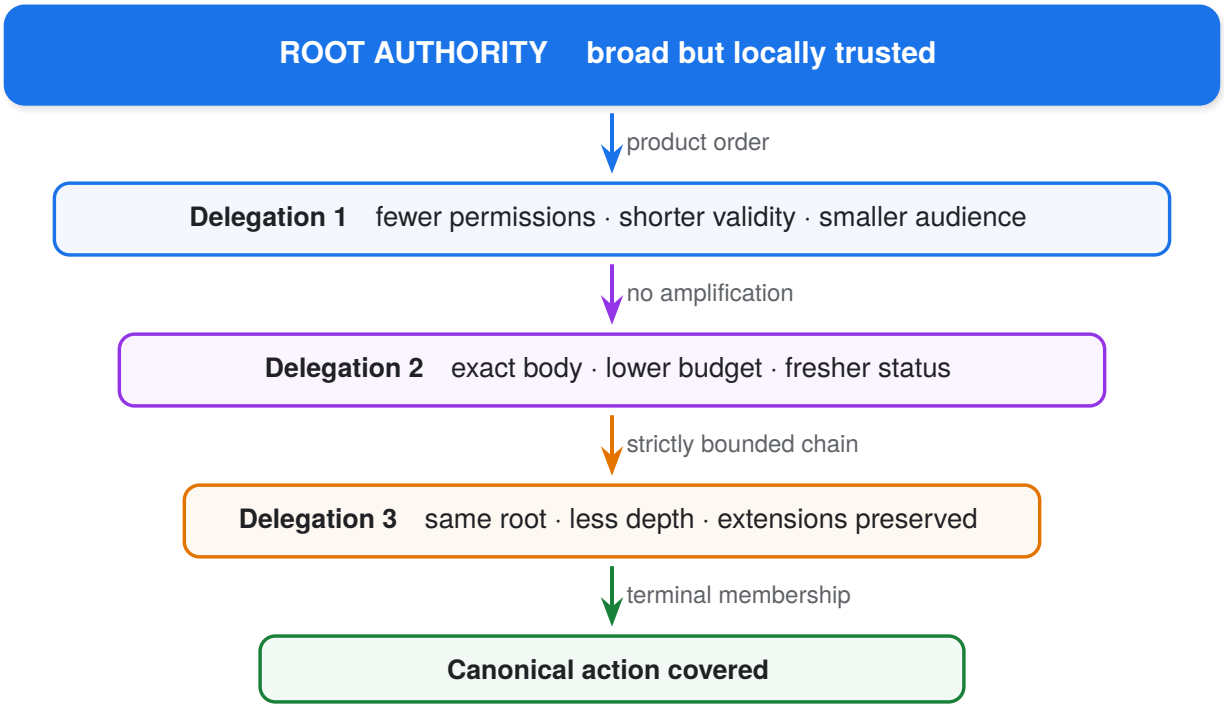

Figure 4: Delegation narrows authority. Every descendant action covered by the terminal scope is also covered by each ancestor. The converse is deliberately false.

The strict depth decrease supplies a termination measure. Exact issuer, subject, parent-grant, and root linkage prevent a valid grant from being moved to another chain.

3.2 Three-valued truth

Each branch produces a truth value

$$\mathbb{T} = \{\bot, ?, \top\}, \qquad \bot \preceq ? \preceq \top,$$

where $\top$ is authorized, $\bot$ is denied, and $?$ is indeterminate. Indeterminate means a recognized trustworthy fact is unavailable while authorization remains reachable. It is never executable authority.

Conjunction and disjunction are minimum and maximum in that order:

$$x \wedge y = \min_{\preceq}(x, y), \qquad x \vee y = \max_{\preceq}(x, y).$$

For a threshold that requires k authorized branches, with a authorized and u indeterminate branches:

$$\text{threshold}(k, a, u) = \begin{cases} \top & a \geq k, \\ ? & a < k \wedge a + u \geq k, \\ \bot & a + u < k. \end{cases}$$

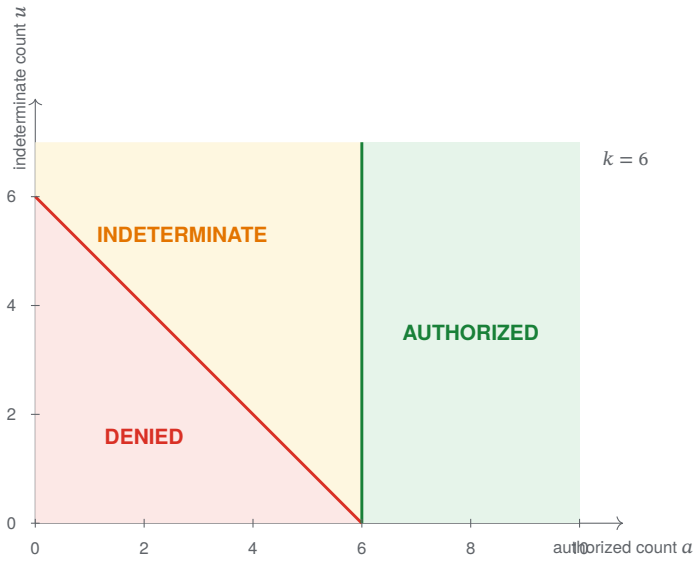

Figure 5: Threshold partition for $k = 6$. Raising the threshold moves the authorization boundary rightward and cannot create authority.

Canonical traversal makes diagnostics deterministic even though the truth algebra is order-independent. A permanent invalid signature may dominate an unavailable status source under conjunction; an unavailable branch may keep a threshold reachable without being counted as authorization.

3.3 Terminal action coverage and budgets

Authorization covers a canonical action only when profile, actor and grant linkage, permission, validity, audience, action-body constraint, status, assurance, critical extensions, and budget all match the terminal authority.

Budget coverage deliberately distinguishes an unbounded authority from an action that fails to state what it may spend:

$$\text{budgetCovers}(c, r) = \begin{cases} \text{true} & c = \text{None}, \\ \text{false} & c = \text{Some}(\_) \wedge r = \text{None}, \\ \text{sameAlgebra}(c, r) \wedge r \leq c & c = \text{Some}(\_) \wedge r = \text{Some}(\_). \end{cases}$$

Thus an absent ceiling is the unbounded top scope. An absent requested budget under a present ceiling is not “zero”; it is an unstated bound and is denied. Profiles that do not have budget semantics issue no ceiling. Profiles that move money or consume bounded capacity derive an explicit requested value in the profile’s own algebra.

TERMINAL COVERAGE RULE

The runtime executes only the canonical action sealed by verification. It never reconstructs provider input from an unverified request, and it never treats an omitted requested budget as covered by a bounded authority.

4. Protocol workflow

4.1 Five verbs and five nouns

The default product surface has five verbs:

$$\{\text{create}, \text{delegate}, \text{execute}, \text{resume}, \text{verify}\}.$$

They act on five product nouns: Identity, Authority, Action, Approval, and Receipt. Signing is a stage of create or delegate, not a separate product verb. Recovery is expressed by resume and status lookup rather than a binding-owned operation.

Create establishes authority from a locally accepted root policy. **Delegate** issues a narrower authority. **Verify** is effect-free and returns the three-valued authorization result. **Execute** admits one exact profile action to the durable lifecycle. **Resume** advances a recoverable workflow using its opaque reference.

4.2 From proposal to sealed command

The complete request path is ordered so that no credential or provider effect precedes authorization and durable intent.

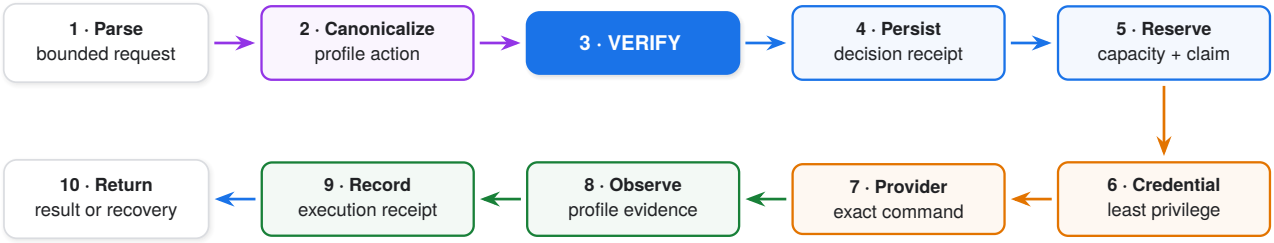

Figure 6: Ordered exact-effect workflow. Configuration equality, authorization, durable decision, reservation, and replay claim all precede credential acquisition and provider entry.

Required and executed verifier configurations are compared before persistence, reservation, credentials, or provider I/O. This prevents a valid proof under one policy from being executed by a runtime configured with another.

4.3 Closed profile routes

There is no generic endpoint containing a profile name plus arbitrary JSON. Every execution route decodes one profile-owned action and invokes one compiled-in service:

```
POST /v1/profiles/opentofu/saved-plan-apply/execute
POST /v1/profiles/postgresql/bounded-update/execute
POST /v1/profiles/github/issue-address/execute
```

Shared runtime code owns identical mechanisms such as durable admission, leases, readiness, and transport. It does not dispatch provider meaning from an operation tag. A new profile begins as a complete vertical: canonical action, policy, evaluator, lifecycle, provider gateway, credential scope, observer, reconciler, receipts, fixtures, and tests. Shared semantic abstractions are extracted only after independent domains prove identical contracts.

5. Durable exact-effect execution

5.1 Client-known recovery before transmission

For every effectful request, the client generates a cryptographically random 32-byte recovery reference before sending any request byte. The runtime stores only its digest and binds it to the canonical request, decision, profile, and operation claim before provider entry.

This ordering solves the response-loss problem. If no request byte was sent, the client can prove non-effect. Once transmission may have occurred, the client already possesses the reference required to query or resume the operation. It never depends on receiving a server-generated token in the response that may be lost.

TRANSPORT FAILURE RULE

Before transmission, failure may be classified as not applied. After transmission of an effectful request, failure is effect-possible until durable workflow state and profile observation establish a stronger result. The SDK returns `reconcile` with the client-known recovery reference; it never recommends a blind retry.

5.2 Durable state machine

Exactly-once behavior is divided into claims the runtime can actually support:

- **exactly-once admission** for one canonical operation reference;
- **atomic reservation and replay claim** inside the lifecycle store;

- **at-most-one active provider attempt** under the workflow lease; and
- **profile-specific convergence** when the provider cannot guarantee exactly-once effects.

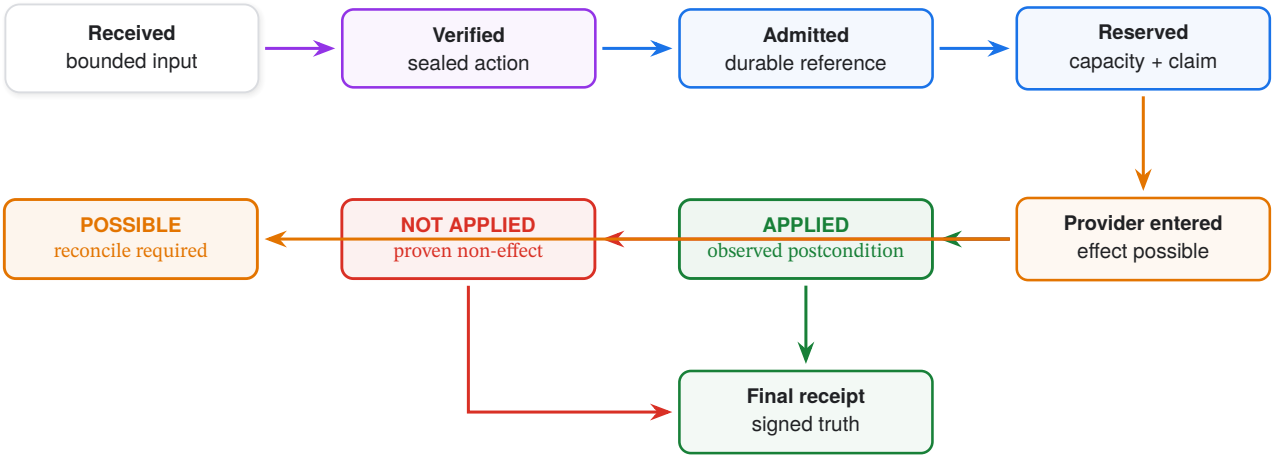

Figure 7: Durable effect state machine. Provider entry makes an effect possible. Only profile-owned observation may refine that state to applied or not applied. Possible outcomes remain recoverable and are never silently released.

Decision, reservation, provider entry, provider result, observation, and receipt persistence are separate durable transitions. A crash before provider entry is provably non-effect. A crash after entry resumes from a possible state and reconciles from fresh evidence.

5.3 Deadlines, cancellation, and admission bounds

An HTTP deadline is not an execution-cancellation guarantee. Effectful work is therefore not launched as untracked blocking work beneath a blanket response timeout. The runtime first acquires a bounded admission permit and commits the workflow. After that point, the durable worker owns progress, independent of the client connection.

Configuration bounds:

- maximum request bytes;
- maximum concurrently admitted operations;
- maximum bounded queue wait;
- per-profile provider concurrency;
- database pool and statement timeouts;
- lease, recovery, and shutdown deadlines; and
- telemetry queue capacity.

Overload is rejected before durable admission and before provider entry with a typed not-applied result. A response deadline after admission returns or projects a recoverable outcome with the known reference. Graceful shutdown stops admission, drains active durable transitions to a safe checkpoint, flushes bounded telemetry, and leaves remaining workflows resumable.

5.4 Failure classification

The product exposes separate axes because one enum cannot answer every operational question.

Axis	Question	Closed values
Authorization truth	Is the action authorized?	denied · indeterminate · authorized
Effect state	What can be claimed about the real effect?	not-applied · possible · applied
Retry class	May the same operation be retried?	never · safe · conditional · unknown
Next call	Which API should the caller invoke?	never · backoff · resume · reconcile
Execution receipt	What did observation establish?	succeeded · failed · indeterminate

An unknown error code fails closed to effect-possible. It does not invent a fourth effect value and does not become not-applied. Stable code, effect, retry, next call, and recommended action remain typed and reachable in every supported language.

6. Profile-owned exact effects

6.1 Why profile semantics remain vertical

Infrastructure, databases, repositories, and payment systems often share a superficial workflow:

```
request -> policy -> credential -> provider -> result
```

The similarity is deceptive. A saved OpenTofu plan is bound to backend state and dependency locks. A PostgreSQL update is bound to a typed statement, predicate, transaction snapshot, and row limit. A GitHub issue mutation is bound to repository identity, issue state, API preconditions, and observable postconditions. Their unknown-outcome and reconciliation rules are not interchangeable.

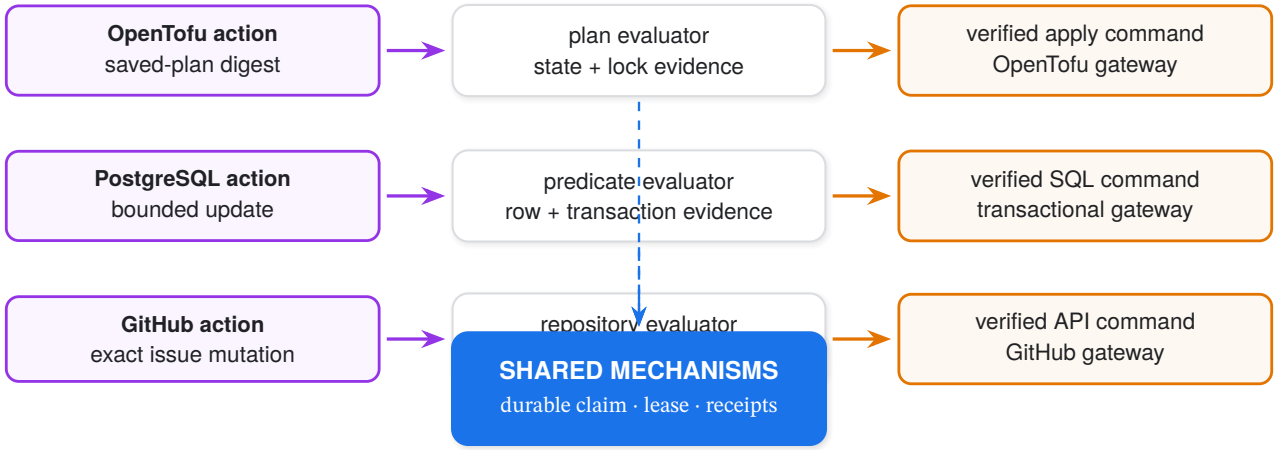

Figure 8: Vertical profile semantics. Each domain owns canonical meaning, evaluator, command, gateway, and observation. Only demonstrated domain-independent mechanisms move into shared runtime packages.

6.2 Credential timing and least privilege

Credentials are acquired only after:

1. canonical decoding and configuration equality;
2. proof verification and profile evaluation;
3. durable decision receipt;
4. atomic reservation and replay claim; and
5. construction of the closed verified command.

The credential scope is derived from the verified command, not supplied by the caller. Provider requests are compared against the command commitment. Credential acquisition failure produces no provider entry. Secrets never appear in canonical fixtures, receipts, telemetry, or errors.

6.3 Reconciliation belongs to the domain

A shared lifecycle may store “possible” and schedule work, but only the profile knows what evidence settles it. OpenTofu may inspect state lineage and plan effects. PostgreSQL may observe transaction or row postconditions. GitHub may compare issue state and idempotency-visible API results.

A reconciler consumes fresh typed evidence and returns one of:

$$\{\text{applied}, \text{not-applied}, \text{still-possible}\}.$$

It cannot reinterpret authorization, broaden the command, or release capacity while the effect remains possible.

7. Open production substrate

7.1 Stock production assembly

The production `auths-node` executable and the reference container use the same closed assembly:

- the verified authority kernel;
- a PostgreSQL lifecycle, workflow, and receipt store over TLS;
- AWS KMS P-256 or PKCS#11 P-256 custody;
- profile-owned OpenTofu, PostgreSQL, and GitHub services;
- durable workflow and receipt ports;
- readiness derived from required dependencies;
- Prometheus and bounded OTLP operations sinks; and
- profile-specific recovery workers.

Production configuration cannot instantiate software fixture custody, in-memory stores, or sandbox providers. The binary fails closed when a required port is absent. The `doctor` command probes the exact assembled dependencies and reports configuration, lifecycle, custody, profiles, and telemetry separately.

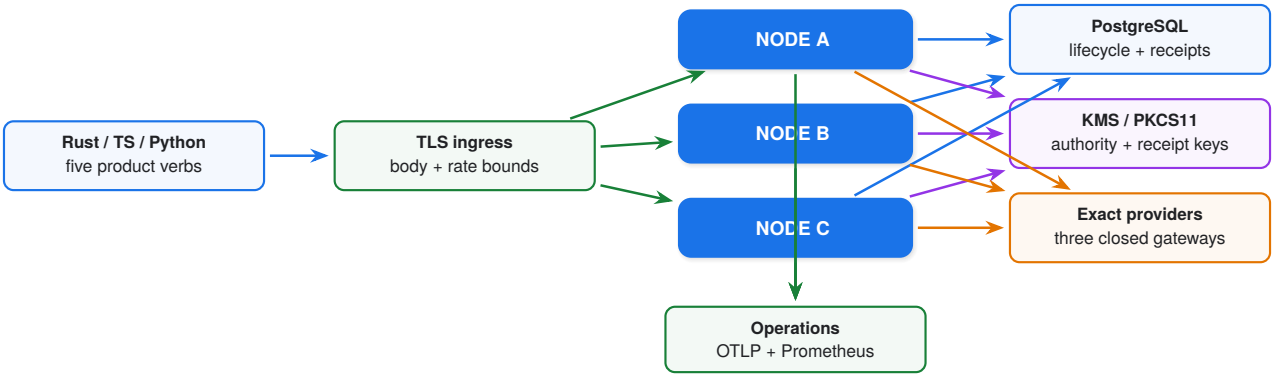

Figure 9: Self-hosted production topology. Three stateless admission nodes share durable lifecycle truth and external custody. Profile providers and operations systems remain explicit dependencies with bounded ports.

7.2 Readiness and dependency policy

Liveness says the process can answer. Readiness says it can safely admit the configured operation set. The node is not ready when lifecycle storage, required custody, trusted context, or an enabled profile's mandatory dependency is unavailable.

Telemetry policy is explicit. Under bounded `DropNewest`, export degradation increments drop/failure counters and appears in doctor and health detail without stopping authorization. A deployment that selects fail-closed audit assurance makes exporter failure affect readiness. No configuration may claim OTLP while silently emitting only Prometheus.

7.3 Bounded operational evidence

Operational events contain closed stages, outcomes, reason codes, durations, and bounded correlation identifiers. They exclude authority bytes, proof contents, commands, provider results, credentials, customer identifiers, and unbounded labels.

The node writes each event to a combined sink:

$$\text{EventSink} = \text{PrometheusProjection} \times \text{BoundedOtlpExporter}\,.$$

The exporter has finite queue capacity, bounded batching, retry/backoff, flush interval, and shutdown deadline. Export success, failure, and drop counts are themselves observable.

8. Receipts and audit

8.1 Decision and execution are separate evidence

A decision receipt records what was authorized: proof commitment, canonical action commitment, trusted-context commitment, profile, decision truth, stable reason, and decision time.

An execution receipt records what happened later: decision receipt identifier, non-reusable execution lease, verified-command digest, outcome, optional result digest, and completion or observation time. Its outcome is `succeeded`, `failed`, or `indeterminate`. Failed asserts non-effect; indeterminate preserves a possible effect.

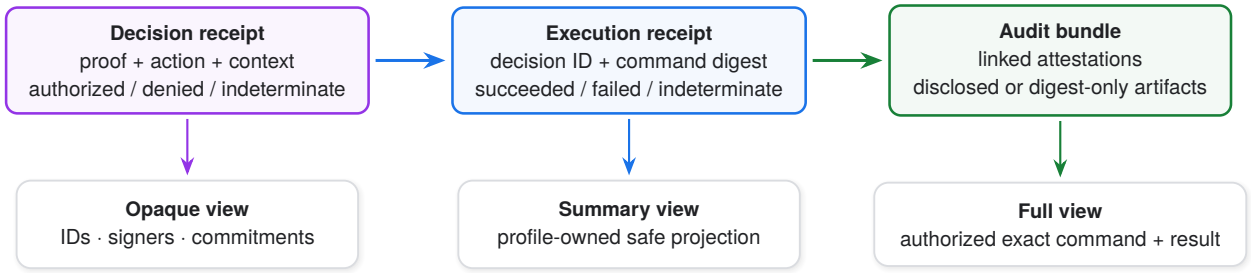

Figure 10: Receipt linkage and disclosure. Authorization evidence and effect evidence are separately signed and content-addressed. Disclosure adds verified detail but never creates authority.

Receipts are signed only after their corresponding gate and durable state are complete. A successful authorization receipt cannot precede replay claim or reservation when the configured contract requires them. An execution receipt never states success when observation remains unknown.

8.2 Canonical attestation

Receipt identifiers are computed from canonical unsigned records. Stored and exported receipts use an attested envelope containing protocol version, receipt kind, canonical bytes, verifier principal, method identifier, signature suite, and signature. Offline verification checks:

1. canonical decoding and content identifier;
2. decision/execution linkage;
3. expected verifier identity and registered signature suite; and
4. disclosed artifact commitments.

Runtime signing uses external custody. An unavailable signer produces no partially attested receipt and, where receipt assurance is fail-closed, prevents execution.

8.3 Bounded disclosure

Receipts remain commitment-only. Exact commands and provider results are stored separately in a bounded disclosure protected by application-owned access control. Inspection offers three inert views:

- **opaque** verifies attestations and returns identifiers, profile, outcome, signers, and commitments;
- **summary** additionally verifies disclosure and returns a bounded profile-owned projection; and
- **full** returns the same projection plus exact verified canonical bytes to a caller already authorized by the application.

Successful inspection is metadata, not a capability, approval, credential, or executable handle.

9. Language surfaces and semantic ownership

Rust owns canonicalization, error classification, lifecycle transitions, receipt meaning, and profile results. WASM and pyo3 are transport layers inside the TypeScript and Python packages. They expose structured envelopes and cannot flatten failures to strings or invent codes.

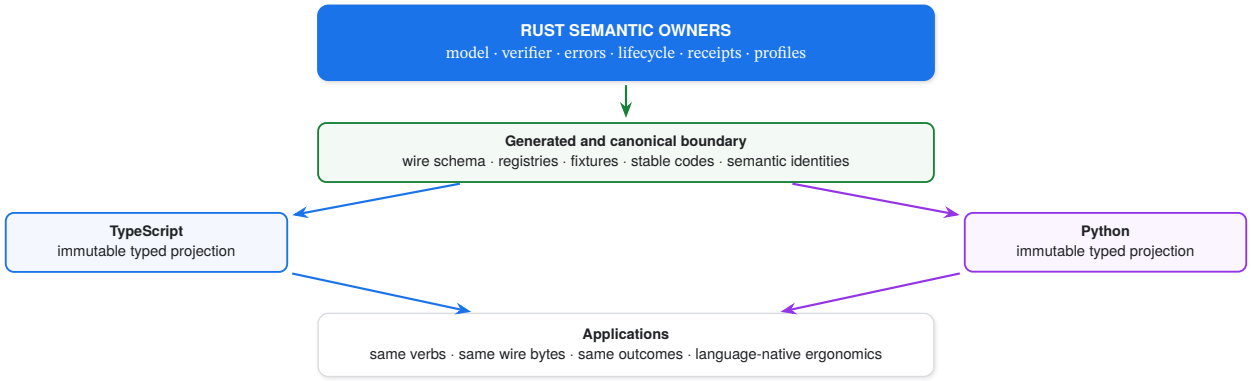

Figure 11: One-way semantic ownership. Bindings may differ ergonomically, but canonical bytes, closed vocabularies, effects, and recovery meaning flow from Rust through generated contracts.

An older client receiving a newer unknown error code preserves it as unknown, classifies effect as possible, and recommends reconciliation where the operation may have crossed transmission. It neither crashes nor silently maps the code to a safe retry.

Independent Go verification is intentionally separate: it is a differential semantic implementation, not another source of protocol truth. Agreement on the canonical corpus increases assurance that neither implementation's local structure has hidden a shared wrapper bug.

10. Formal and executable assurance

10.1 Readable semantics and shipping Rust

Lean 4 specifies rich authority values, component orders, delegation transitions, terminal coverage, three-valued composition, and monotonicity (Moura and Ullrich 2021). The model uses finite sets, intervals, constraints, and natural-number bounds rather than Rust allocation or wire layout.

Production authoring, delegation, and coverage are pure, total, `unsafe`-free Rust decisions over constructor-validated borrowed views. Charon lowers those exact functions; Aeneas translates them to Lean (Ho and Protzenko 2022). Representation bridges relate translated carriers and predicates to the readable model. Refinement theorems establish that the production decisions agree with the rich semantics under explicit representation premises.

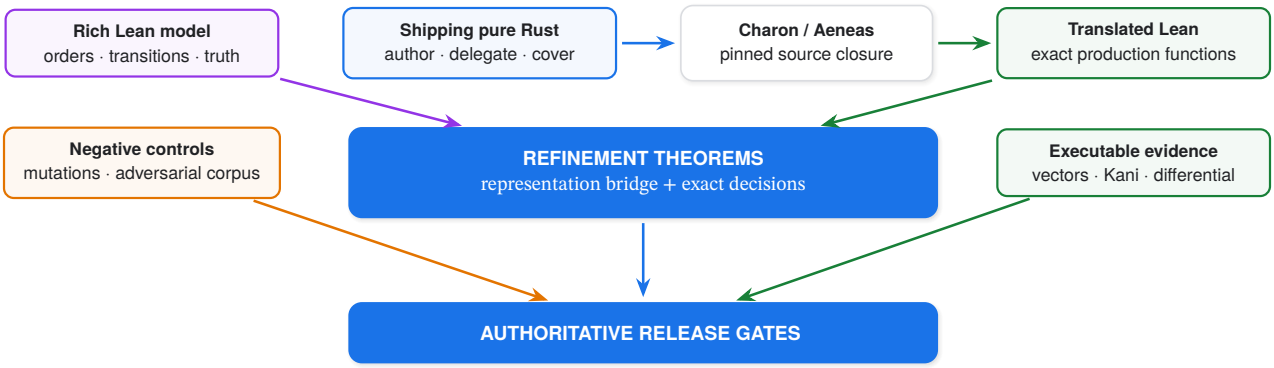

Figure 12: Assurance chain. The model is readable, the translated functions are the shipping decisions, and independent negative and bounded evidence surrounds the refinement proof.

10.2 Generated and bounded evidence

A versioned algebra contract generates the Rust and Lean attenuation projection and threshold classifier. Normal verification regenerates both and rejects byte drift. Lean exports canonical semantic vectors that shipping Rust replays. The finite Boolean attenuation surface and configured threshold domain are exhaustively enumerated.

Kani checks bit-precise properties over shipping Rust (Delmas et al. 2026; Kroening, Schrammel, and Tautschnig 2023). Every package containing a registered proof harness is included in the formal gate, and the harness inventory

rejects new ungated roots. Mutation controls deliberately break security-relevant branches and require the corresponding harness or vector to fail. A passing proof harness that cannot be reddened by a relevant mutation is not accepted as evidence.

10.3 Conformance as a negative claim

Positive canonical vectors show that valid cases agree. Adversarial conformance shows that malformed, widened, stale, misconfigured, or incorrectly linked cases fail with the required stable result. The adversarial corpus is part of standalone compliance, aggregate CI, and release checking. A controlled expected-result mutation must make each aggregate path red.

Cross-language checks require Rust, TypeScript, Python, WASM, and the independent Go verifier to agree where their declared roles overlap. Error registry and lifecycle metadata are exact sets, so a new public code cannot exist without an active compatibility disposition.

10.4 Reproducible release evidence

The assurance manifest binds claims to:

- compiled theorem names and statement hashes;
- transitive axioms and residual assumptions;
- source and translation closure;
- pinned Rust, Lean, Charon, Aeneas, Kani, and CBMC versions;
- generated artifact hashes;
- canonical and adversarial corpus results;
- language and installed-package conformance;
- production topology and profile identities; and
- retained fault, load, recovery, and operations evidence.

Semantic identities make change control part of security. A changed meaning must receive a reviewed version assignment before freeze metadata is updated. Release gates fail when implementation meaning, formal meaning, fixtures, registries, or public support metadata drift independently.

11. Security consequences

11.1 Attacks and defenses

Attack or failure	Security consequence	Enforced response
Authenticated actor without authority	Identity is mistaken for permission	Kernel requires a valid locally trusted delegation chain
Widened child grant	Delegation amplification	Eleven-dimensional product order denies the edge
Missing requested budget below a ceiling	Unbounded spend hides under omission	Terminal coverage denies absent requested budget
Valid proof under different configuration	Policy confusion	Required/executed mismatch stops before state or I/O
Duplicate execution request	Repeated external effect	Client reference, atomic claim, reservation, and profile idempotency
Timeout after request transmission	Blind retry may duplicate effect	Effect-possible result with known recovery reference
Crash after provider entry	Capacity released while effect may exist	Durable possible state retained until reconciliation
Forged or cross-linked receipt	False audit claim	Canonical ID, signature, signer, and linkage verification
Unknown error code in an older SDK	Unsafe fallback	Preserve code, effect possible, reconcile

Attack or failure	Security consequence	Enforced response
Telemetry outage	Silent loss of evidence	Bounded policy, counters, doctor detail, optional fail-closed readiness
Adversarial corpus drift	Negative behavior silently changes	Aggregate compliance and release gates fail

11.2 Bounded work

Attacker-controlled inputs are bounded in bytes, collection sizes, graph nodes, chain depth, recursion, plan leaves, cryptographic work, database records, queue capacity, concurrent operations, and provider concurrency. Parsing produces closed domain types before semantics run. Unknown, unsupported, oversized, duplicate, non-canonical, and ambiguous values fail closed without panics.

The Lean structural-cost theorem describes the declared algebraic traversal, not total empirical runtime. Allocation, hashing, signatures, database I/O, provider latency, and exporter work have separate implementation bounds and qualification evidence.

11.3 Change control

SEMANTIC CHANGE RULE

A new authority dimension, truth value, effect state, profile outcome, or recovery transition must land as one reviewable change across Rust ownership, Lean meaning, generated contracts, translation closure, fixtures, language projections, registries, adversarial controls, semantic identities, and release evidence.

This prevents a production check from appearing without a formal counterpart, a binding from inventing vocabulary, a formal theorem from proving a superseded predicate, or a release gate from omitting the negative corpus it claims to enforce.

12. Assurance boundary and limitations

> **Auths-Proof proves and tests a bounded claim, not universal provider correctness.** It establishes exact authorization semantics, refinement of isolated production decisions, durable admission and recovery contracts, closed profile orchestration, and release-bound conformance. External systems remain nondeterministic and are handled through observation and reconciliation rather than assumed atomic.

The principal residual boundaries are:

Model adequacy. A correct theorem about the wrong authority model remains wrong. The closed dimensions, profile meanings, and trust policy require human security review.

Representation and cryptography. Refinement starts from validated Rust views. Canonical decoding, constructor invariants, signature libraries, principal adapters, and key custody have separate conformance and trusted implementation boundaries.

Qualified deployments. The production claim applies to named topology, store, custody adapters, profiles, limits, SDKs, and release artifacts. It does not automatically transfer to modified deployments or unregistered profiles.

External observations. A provider may expose insufficient evidence to resolve a possible effect. The safe result can remain indeterminate indefinitely; availability does not justify inventing certainty.

Toolchain basis. Lean, Mathlib, Charon, Aeneas, Rust, Kani, CBMC, build infrastructure, and reviewed external models remain part of the trusted evidence pipeline.

Application disclosure policy. Auths verifies receipt disclosure and returns inert views. The embedding application still authenticates the viewer, owns encryption and retention, and decides who may request summary or full detail.

These limitations are explicit because authorization assurance is meaningful only when the boundary of the claim is as precise as the claim itself.

13. Related work

Capability systems made authority an explicit transferable object (Dennis and Horn 1966). SPKI and SDSI bind authorization to keys and local names (Ellison et al. 1999; Rivest and Lampson 1996). Macaroons provide efficient attenuation through contextual caveats (Birgisson et al. 2014). PolicyMaker, KeyNote, and RT separate assertions, local policy, and compliance checking (Blaze, Feigenbaum, and Lacy 1996; Blaze et al. 1999; Li, Mitchell, and Winsborough 2002).

Auths-Proof shares explicit delegation and verifier-local policy, but uses a closed multidimensional authority order rather than a general-purpose policy language. The restriction supports deterministic canonicalization, formal containment, exhaustive finite projections, and profile-owned exact effects.

Proof-carrying code moves the burden of supplying safety evidence toward the producer (Necula 1997). Proof-carrying authentication applies the idea to distributed authorization judgments (Appel and Felten 1999; Bauer, Schneider, and Felten 2002). Auths authorization proofs are protocol evidence, not Lean proof terms; the receiver still runs a fixed verifier under local trust.

CompCert and seL4 demonstrate the strength of explicit refinement chains (Leroy 2009; Klein et al. 2009). Translation validation checks a particular translated artifact rather than proving a translator for all inputs (Pnueli, Siegel, and Singerman 1998). Auths-Proof combines qualified functional translation of safe Rust with handwritten refinement theorems, generated finite contracts, and independent executable controls. Rust language safety narrows memory-safety risk but remains distinct from authorization correctness (Jung et al. 2018).

Property-based testing and fuzzing explore boundaries excluded from the mathematical model (Claessen and Hughes 2000; Miller, Fredriksen, and So 1990). They complement theorems, exhaustive enumeration, bounded model checking, adversarial conformance, and production fault tests; none is relabelled as another kind of evidence.

14. Conclusion

Auths-Proof treats authorization as a complete systems problem rather than a middleware predicate. Portable evidence is useful only under local trust. Delegation is useful only when it cannot amplify authority. A successful decision is useful only when the executor consumes the exact sealed action. And external automation is safe only when crashes, timeouts, response loss, and provider ambiguity remain recoverable and honestly recorded.

The architecture therefore has a small semantic center and an explicit operational perimeter. The offline kernel decides three-valued authorization. The durable runtime admits one canonical effect under a client-known recovery reference. Profile-owned services derive least-privilege commands and reconcile provider-specific outcomes. Separate decision and execution receipts preserve what was authorized and what was observed. Operations surfaces expose bounded health without leaking authority. Language bindings project one Rust-owned contract. Formal refinement and adversarial release gates keep those layers from drifting apart.

> **An Auths authority permits one exact action under explicit constraints; the production runtime either proves non-effect, proves the observed effect, or retains a durable recovery path without inventing certainty.**

That is the central technical claim: not that distributed effects become magically atomic, but that authorization, execution, uncertainty, and evidence remain separate, bounded, and mechanically connected from proof to provider.

References

Appel, Andrew W., and Edward W. Felten. 1999. “Proof-Carrying Authentication.” In *Proceedings of the 6th ACM Conference on Computer and Communications Security*, 52–62. https://doi.org/10.1145/319709.319718.

Bauer, Lujo, Michael A. Schneider, and Edward W. Felten. 2002. “A General and Flexible Access-Control System for the Web.” In *Proceedings of the 11th USENIX Security Symposium*, 93–108. USENIX Association. https://www.usenix.org/legacy/events/sec02/full_papers/bauer/bauer_html/.

Birgisson, Arnar, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. 2014. “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud.” In *Network and Distributed System Security Symposium*. Internet Society. https://research.google/pubs/macaroons-cookies-with-contextual-caveats-for-decentralized-authorization-in-the-cloud/.

Blaze, Matt, Joan Feigenbaum, John Ioannidis, and Angelos D. Keromytis. 1999. “The KeyNote Trust-Management System Version 2.” RFC 2704. RFC Editor. https://doi.org/10.17487/RFC2704.

Blaze, Matt, Joan Feigenbaum, and Jack Lacy. 1996. “Decentralized Trust Management.” In *Proceedings of the 1996 IEEE Symposium on Security and Privacy*, 164–73. https://doi.org/10.1109/SECPRI.1996.502679.

Claessen, Koen, and John Hughes. 2000. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 268–79. https://doi.org/10.1145/351240.351266.

Delmas, Rémi, Zyad Hassan, Qinheping Hu, Rahul Kumar, Felipe R. Monteiro, Thanh Nguyen, Adrián Palacios, et al. 2026. “Kani: A Model Checker for Rust.” *arXiv Preprint arXiv:2607.01504*. https://doi.org/10.48550/arXiv.2607.01504.

Dennis, Jack B., and Earl C. Van Horn. 1966. “Programming Semantics for Multiprogrammed Computations.” *Communications of the ACM* 9 (3): 143–55. https://doi.org/10.1145/365230.365252.

Ellison, Carl M., Bill Frantz, Butler Lampson, Ron Rivest, Brian M. Thomas, and Tatu Ylonen. 1999. “SPKI Certificate Theory.” RFC 2693. RFC Editor. https://doi.org/10.17487/RFC2693.

Ho, Son, and Jonathan Protzenko. 2022. “Aeneas: Rust Verification by Functional Translation.” *Proceedings of the ACM on Programming Languages* 6 (ICFP): 711–41. https://doi.org/10.1145/3547647.

Jung, Ralf, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018. “RustBelt: Securing the Foundations of the Rust Programming Language.” *Proceedings of the ACM on Programming Languages* 2 (POPL): 66:1–34. https://doi.org/10.1145/3158154.

Klein, Gerwin, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, et al. 2009. “seL4: Formal Verification of an OS Kernel.” In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles*, 207–20. https://doi.org/10.1145/1629575.1629596.

Kroening, Daniel, Peter Schrammel, and Michael Tautschnig. 2023. “CBMC: The C Bounded Model Checker.” *arXiv Preprint arXiv:2302.02384*. https://doi.org/10.48550/arXiv.2302.02384.

Leroy, Xavier. 2009. “Formal Verification of a Realistic Compiler.” *Communications of the ACM* 52 (7): 107–15. https://doi.org/10.1145/1538788.1538814.

Li, Ninghui, John C. Mitchell, and William H. Winsborough. 2002. “Design of a Role-Based Trust-Management Framework.” In *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, 114–30. https://doi.org/10.1109/SECPRI.2002.1004376.

Miller, Barton P., Lars Fredriksen, and Bryan So. 1990. “An Empirical Study of the Reliability of UNIX Utilities.” *Communications of the ACM* 33 (12): 32–44. https://doi.org/10.1145/96267.96279.

Moura, Leonardo de, and Sebastian Ullrich. 2021. “The Lean 4 Theorem Prover and Programming Language.” In *Automated Deduction – CADE 28*, 12699:625–35. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-030-79876-5_37.

Necula, George C. 1997. “Proof-Carrying Code.” In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 106–19. https://doi.org/10.1145/263699.263712.

Pnueli, Amir, Michael Siegel, and Eli Singerman. 1998. “Translation Validation.” In *Tools and Algorithms for the Construction and Analysis of Systems*, 1384:151–66. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/BFb0054170.

Rivest, Ronald L., and Butler Lampson. 1996. “SDSI: A Simple Distributed Security Infrastructure.” Technical memo. Massachusetts Institute of Technology. https://www.microsoft.com/en-us/research/publication/sdsi-a-simple-distributed-security-infrastructure/.